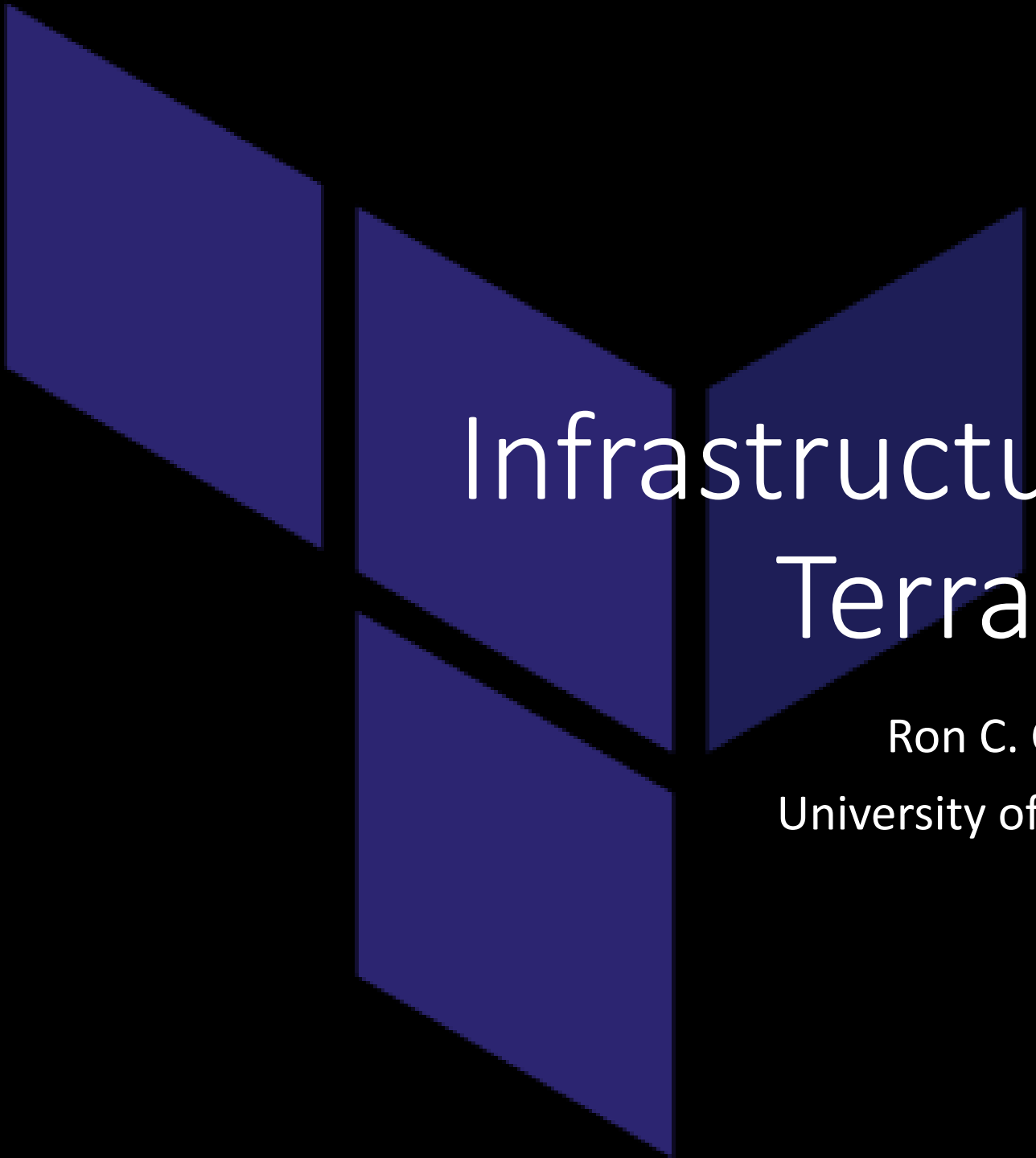


Infrastructure as Code Terraform

Ron C. Chiang

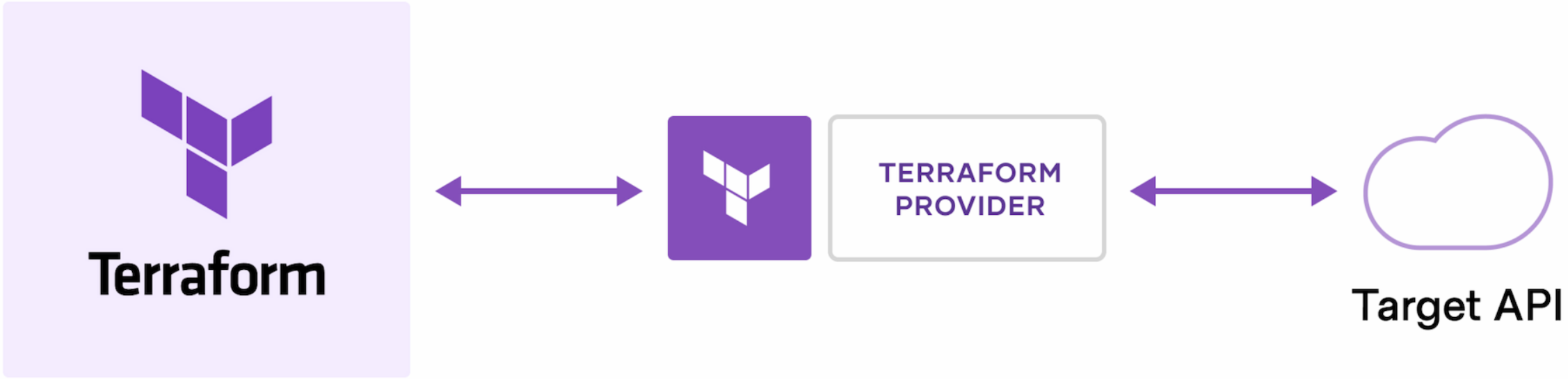
University of St. Thomas

What is Terraform?

- An infrastructure as code tool that lets you define both cloud and on-prem resources in human-readable, declarative configuration files that you can version, reuse, and share.

- Terraform can manage low-level components like compute, storage, and networking resources, as well as high-level components like DNS entries and SaaS features.

- Allow users to use a consistent *workflow* to provision and manage infrastructure throughout its lifecycle



How does Terraform work?

- Terraform creates and manages resources on cloud platforms and other services through their application programming interfaces (APIs).
- *Providers* enable Terraform to work with virtually any platform or service with an accessible API.
- You can find all publicly available *providers* on the Terraform Registry, including Amazon Web Services (AWS), Azure, Google Cloud Platform (GCP), Kubernetes, Helm, GitHub, Splunk, DataDog, and many more.

Core Terraform Workflow – three stages

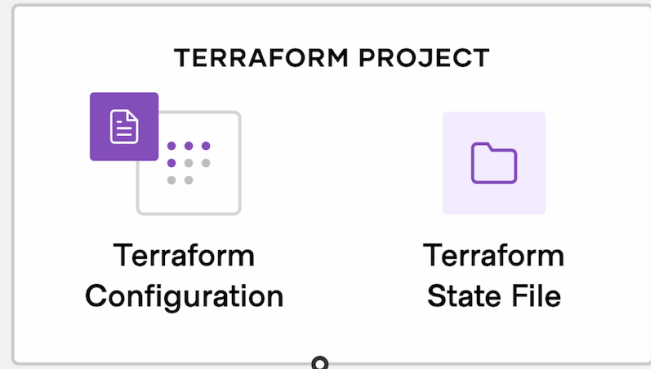
- **Write**: You define resources, which may be across multiple cloud providers and services. For example, you might create a configuration to deploy an application on virtual machines in a Virtual Private Cloud (VPC) network with security groups and a load balancer.

- **Plan**: Terraform creates an execution plan describing the infrastructure it will create, update, or destroy based on the existing infrastructure and your configuration.

- **Apply**: On approval, Terraform performs the proposed operations in the correct order, respecting any resource dependencies. For example, if you update the properties of a VPC and change the number of virtual machines in that VPC, Terraform will recreate the VPC before scaling the virtual machines.

Write

Define infrastructure in configuration files



Plan

Review the changes Terraform will make to your infrastructure

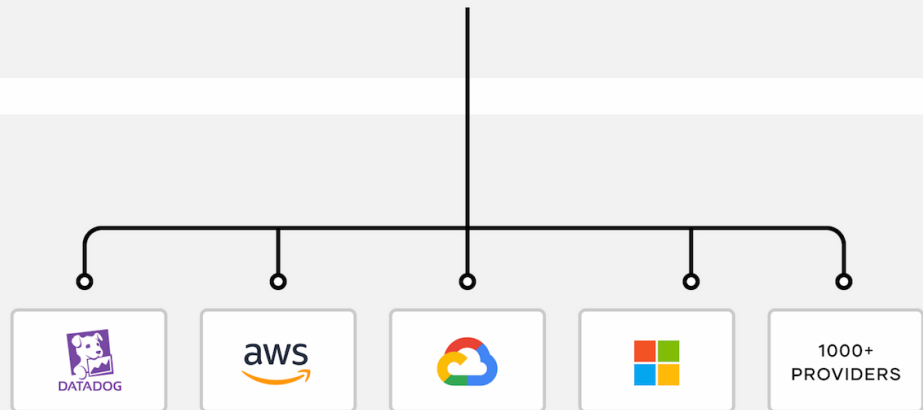
```
$ terraform plan

...

Terraform will perform
the following actions
```

Apply

Terraform provisions your infrastructure and updates the state file.



Terraform's advantages over Hand-crank Cloud using GUI console



Terraform can manage infrastructure on multiple cloud platforms.



The human-readable configuration language helps you write infrastructure code quickly.



Terraform's state allows you to track resource changes throughout your deployments.



You can commit your configurations to version control to safely collaborate on infrastructure.

Benefits

- Manage any infrastructure
  - Terraform plugins called *providers* let Terraform interact with cloud platforms and other services via their application programming interfaces (APIs).
  - Find *providers* for many of the platforms and services you already use in the Terraform Registry.
  - Terraform takes an *immutable* approach to infrastructure, reducing the complexity of upgrading or modifying your services and infrastructure.

Benefits

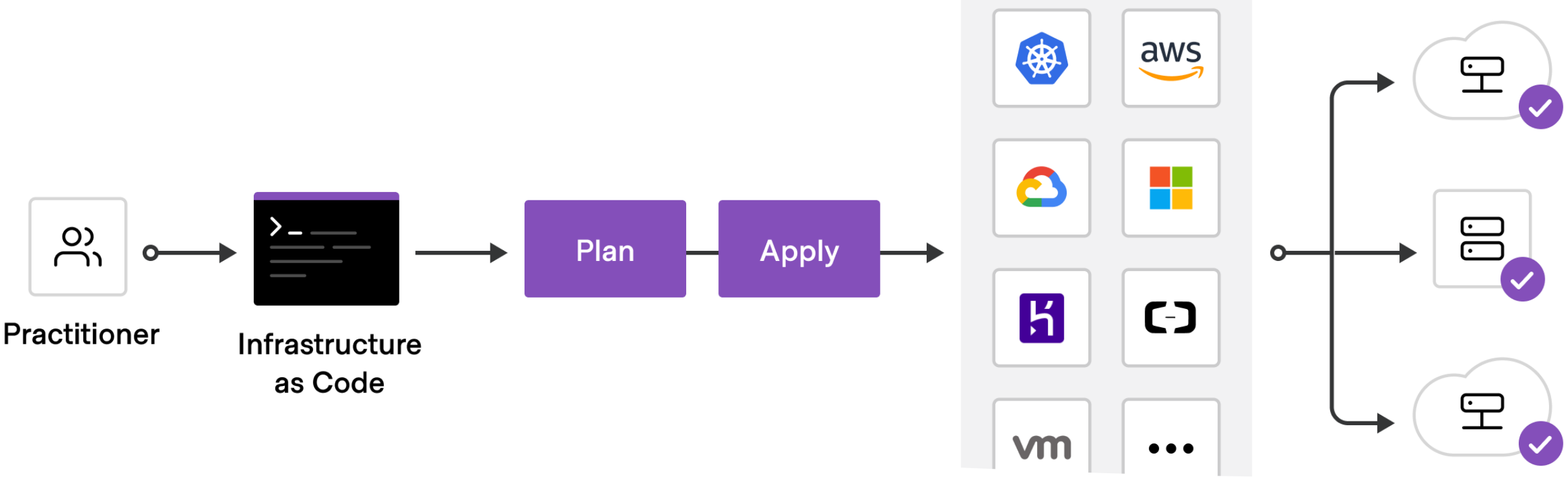
- Track your infrastructure
  - Terraform generates a plan and prompts you for your approval before modifying your infrastructure.
  - It also keeps track of your real infrastructure in a *state* file, which acts as a source of truth for your environment.
  - Terraform uses the state file to determine the changes to make to your infrastructure so that it will match your configuration.

Benefits

- Automate changes and workflow
  - Terraform configuration files are *declarative*, meaning that they describe the end state of your infrastructure.
  - In contrast to procedural programming languages that requireto write step-by-step instructions to create resources, Terraform handles the underlying logic. Terraform providers automatically calculate dependencies between resources to create or destroy them in the correct order.
  - Terraform builds a resource graph to determine resource dependencies and creates or modifies non-dependent resources in parallel. This allows Terraform to provision resources efficiently.

Benefits

- Standardize configurations
  - Terraform supports reusable configuration components called *modules* that define configurable collections of infrastructure, saving time and encouraging best practices.
  - You can use publicly available modules from the Terraform Registry, or write your own.



To deploy infrastructure with Terraform

- **Scope** - Identify the infrastructure for your project.
- **Author** - Write the configuration for your infrastructure.
- **Initialize** - Install the plugins Terraform needs to manage the infrastructure.
- **Plan** - Preview the changes Terraform will make to match your configuration.
- **Apply** - Make the planned changes.



Collaborate

- Terraform allows you to collaborate on your infrastructure with its remote state backends. When you use Terraform Cloud (free for up to five users), you can securely share your state with your teammates, provide a stable environment for Terraform to run in, and prevent race conditions when multiple people make configuration changes at once.
- You can also connect Terraform Cloud to version control systems (VCSs) like GitHub, GitLab, and others, allowing it to automatically propose infrastructure changes when you commit configuration changes to VCS. This lets you manage changes to your infrastructure through version control, as you would with application code.

Install Terraform on Amazon Linux

- Create and Open a Cloud9 environment
- Install yum-config-manager to manage your repositories
  - `sudo yum install -y yum-utils`
- Use yum-config-manager to add the official HashiCorp Linux repository
  - `sudo yum-config-manager --add-repo https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo`
- Install Terraform from the new repository
  - `sudo yum -y install terraform`

Verify the installation

- Verify that the installation worked by opening a new terminal session and listing Terraform's available subcommands. For example,
  - terraform -help

- Add any subcommand to terraform -help to learn more about what it does and available options. For example,
  - terraform -help plan

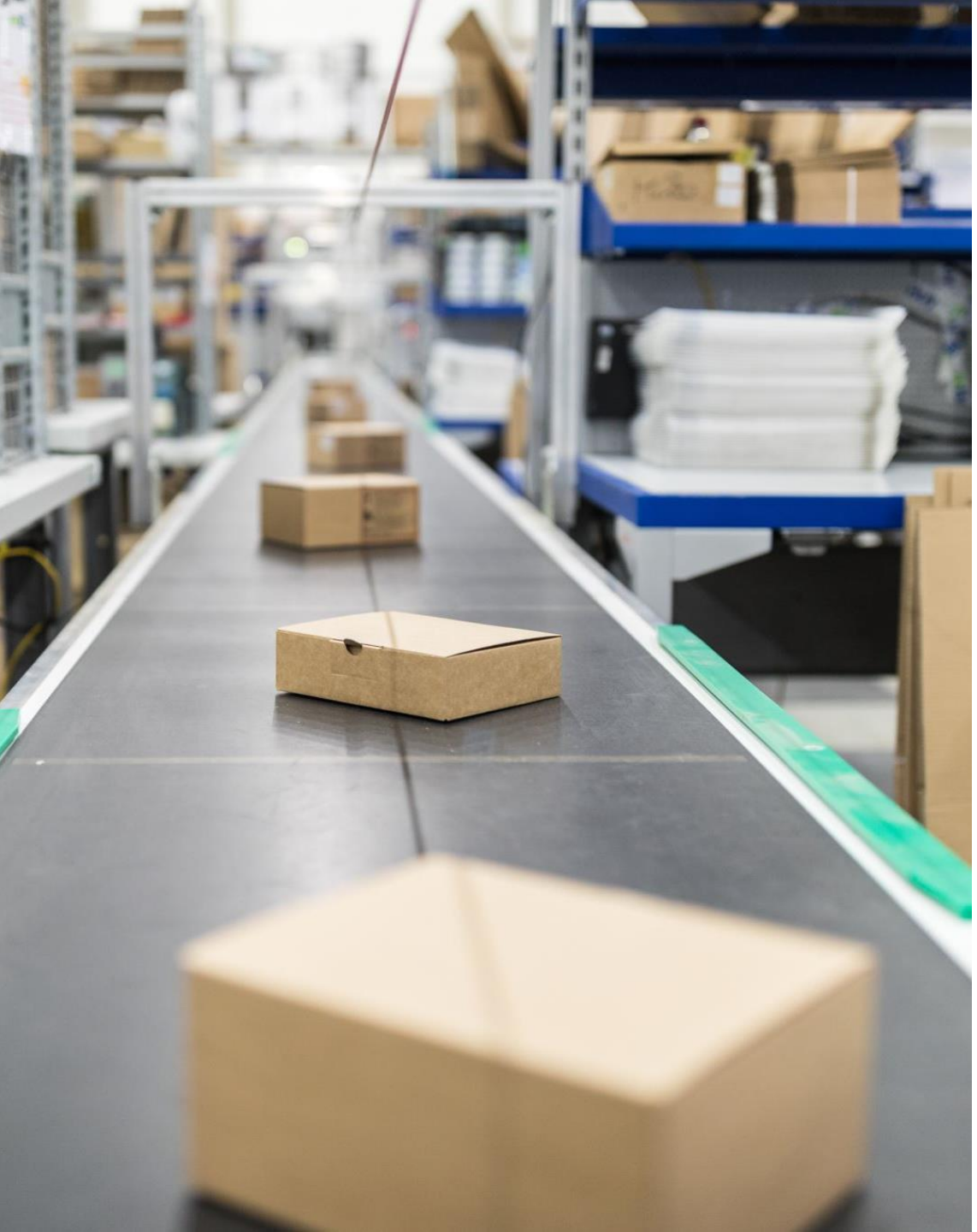
Terraform LAB: Deploy a Webserver Container

- **Goal:** Provision an NGINX server using Docker on Linux
- Cloud9 should have docker engine. Use `docker --version` to verify
- Create a directory named `terraform-lab-docker-container`
- Navigate into `terraform-lab-docker-container` directory



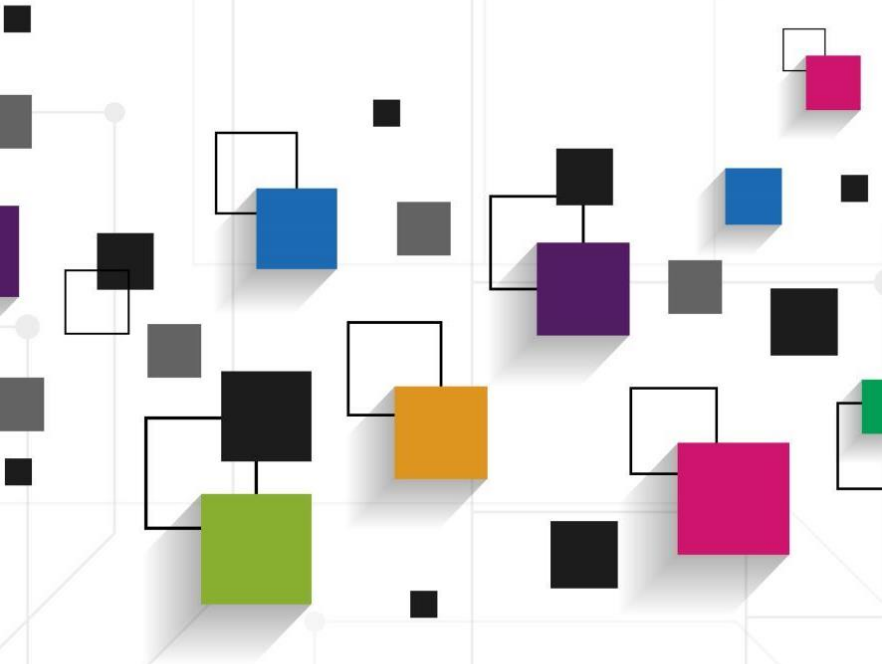
Terraform LAB: Deploy a Webserver Container

- Author
  - In the working directory, create a Terraform configuration file called `main.tf`
    - https://github.com/rclc/terraform-lab-docker-container/blob/master/main.tf
- Init
  - Initialize the project, which downloads a plugin called a provider that lets Terraform interact with Docker
    - `terraform init`
- Plan and Apply
  - Provision the NGINX server container with apply. When Terraform asks you to confirm type yes and press ENTER.
    - `terraform apply`



Terraform LAB: Deploy a Webserver Container

- Verify
  - Verify the existence of the NGINX container by
    - `curl http://localhost:8000`
  - Or, running `docker ps` to see the container.
- Cleanup
  - To stop the container, run `terraform destroy`



Terraform Configuration Language

- The main purpose of the Terraform language is declaring resources, which represent infrastructure objects. All other language features exist only to make the definition of resources more flexible and convenient.
- Configuration files you write in Terraform language tell Terraform what plugins to install, what infrastructure to create, and what data to fetch.
- Terraform language also lets you define dependencies between resources and create multiple similar resources from a single configuration block.

Terraform Configuration

- A Terraform *configuration* is a complete document in the Terraform language that tells Terraform how to manage a given collection of infrastructure.
- A configuration can consist of *multiple files and directories*



Terraform Language Syntax

- *Blocks* are containers for other content and usually represent the configuration of some kind of object, like a resource.
- Blocks have a *block type*, can have zero or more *labels*, and have a *body* that contains any number of arguments and nested blocks.
- Most of Terraform's features are controlled by top-level blocks in a configuration file.

```
resource "aws_vpc" "main" {
  cidr_block = var.base_cidr_block
}
```

```
<BLOCK TYPE> "<BLOCK LABEL>" "<BLOCK LABEL>" {
  # Block body
  <IDENTIFIER> = <EXPRESSION> # Argument
}
```

Terraform Language Syntax

- *Arguments* assign a value to a name/ID. They appear within blocks.
- *Expressions* represent a value, either literally or by referencing and combining other values. They appear as values for arguments, or within other expressions.

```
resource "aws_vpc" "main" {
  cidr_block = var.base_cidr_block
}
```

```
<BLOCK TYPE> "<BLOCK LABEL>" "<BLOCK LABEL>" {
  # Block body
  <IDENTIFIER> = <EXPRESSION> # Argument
}
```

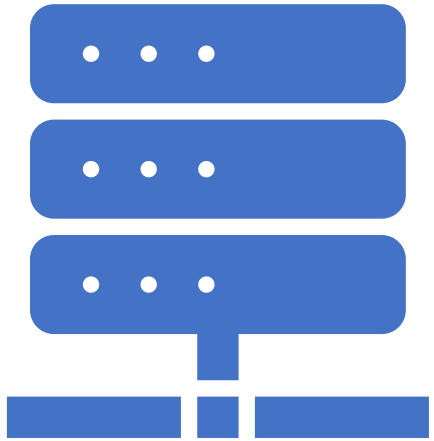
Some Properties of Terraform Language

- The Terraform language is declarative, describing an intended goal rather than the steps to reach that goal.

- The ordering of blocks and the files they are organized into are generally not significant; Terraform only considers implicit and explicit relationships between resources when determining an order of operations.

- Let's use the next lab to learn more about the Terraform Language.

Terraform Lab: AWS Instance



- This lab will provision an EC2 instance on Amazon Web Services

- Prerequisites
  - The Terraform CLI (1.2.0+), which is installed in the previous lab
  - The AWS CLI, which is installed in Cloud9 instance by default
  - AWS account and associated credentials that allow you to create resources (We don't need to configure this here because we use Cloud9 as AWS account admin)

- Create and navigate into a directory for the project
  - `mkdir terraform-lab-aws-instance`
  - `cd terraform-lab-aws-instance`

- Create a file called `main.tf` and follow the next few pages to write terraform configuration

Write Configuration – terraform Block

- The *terraform {}* block contains Terraform settings, including the required providers Terraform will use to provision your infrastructure.

- For each provider, the source attribute defines an optional hostname, a namespace, and the provider type.

- Terraform installs providers from the *Terraform Registry* by default. In this example configuration, the `aws` provider's source is defined as `hashicorp/aws`, which is shorthand for `registry.terraform.io/hashicorp/aws`.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 4.16"
    }
  }

  required_version = ">= 1.2.0"
}
```

Write Configuration – terraform Block

- You can also set a version constraint for each provider defined in the `required_providers` block.
- The version attribute is optional, but we recommend using it to constrain the provider version so that Terraform does not install a version of the provider that does not work with your configuration.
- If you do not specify a provider version, Terraform will automatically download the most recent version during initialization.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 4.16"
    }
  }

  required_version = ">= 1.2.0"
}
```

Write Configuration – provider Block

- The *provider* block configures the specified provider, in this case *aws*. A provider is a plugin that Terraform uses to create and manage your resources.

- You can use multiple provider blocks in your Terraform configuration to manage resources from different providers.

- You can even use different providers together. For example, you could pass the IP address of your AWS EC2 instance to a monitoring resource from DataDog.

```
provider "aws" {
  region  = "us-west-2"
}
```

Write Configuration – resource Block

- Use *resource* blocks to define components of your infrastructure.
- A resource might be a physical or virtual component such as an EC2 instance, or it can be a logical resource such as a Heroku application.

```
resource "aws_instance" "app_server" {
  ami = "ami-830c94e3"
  instance_type = "t2.micro"

  tags = {
    Name = "ExampleAppServerInstance"
  }
}
```

Write Configuration – resource Block

- Resource blocks have two strings before the block: the *resource type* and the *resource name*.
- In this example, the resource type is `aws_instance` and the name is `app_server`.
- The prefix of the type maps to the name of the provider. In the example configuration, Terraform manages the `aws_instance` resource with the `aws` provider.
- Together, the resource type and resource name form a unique ID for the resource. For example, the ID for your EC2 instance is `aws_instance.app_server`.

```
resource "aws_instance" "app_server" {
  ami = "ami-830c94e3"
  instance_type = "t2.micro"

  tags = {
    Name = "ExampleAppServerInstance"
  }
}
```

Write Configuration – resource Block

- Resource blocks contain *arguments* which you use to configure the resource.
- Arguments can include things like machine sizes, disk image names, or VPC IDs.
- The *providers reference* lists the required and optional arguments for each resource.
- The example configuration sets the AMI ID to an Ubuntu image, and the instance type to t2.micro, which qualifies for AWS' free tier. It also sets a tag to give the instance a name.

```
resource "aws_instance" "app_server" {
  ami = "ami-830c94e3"
  instance_type = "t2.micro"

  tags = {
    Name = "ExampleAppServerInstance"
  }
}
```

Initialize the directory

- When you create a new configuration — or check out an existing configuration from version control — you need to initialize the directory with `terraform init`.
- Initializing a configuration directory downloads and installs the providers defined in the configuration, which in this case is the `aws` provider.
- Terraform downloads the `aws` provider and installs it in a hidden subdirectory of your current working directory, named `.terraform`.
- The `terraform init` command prints out which version of the provider was installed. Terraform also creates a lock file named `.terraform.lock.hcl` which specifies the exact provider versions used, so that you can control when you want to update the providers used for your project.

Format and validate the configuration

- Use consistent formatting in all of your configuration files. The `terraform fmt` command automatically updates configurations in the current directory for readability and consistency.
- Terraform will print out the names of the files it modified, if any.

- You can also make sure your configuration is syntactically valid and internally consistent by using the `terraform validate` command.
- The example configuration provided above is valid, so Terraform will return a success message.

Create infrastructure

- Apply the configuration now with the `terraform apply` command
- Before it applies any changes, Terraform prints out the *execution plan* which describes the actions Terraform will take to change your infrastructure to match the configuration.
- The output has a + next to aws_instance.app_server, meaning that Terraform will create this resource. Beneath that, it shows the attributes that will be set. When the value displayed is (known after apply), it means that the value will not be known until the resource is created.
- Terraform will now pause and wait for your approval before proceeding. If anything in the plan seems incorrect or dangerous, it is safe to abort here before Terraform modifies your infrastructure.
- In this case the plan is acceptable, so type yes at the confirmation prompt to proceed. Executing the plan will take a few minutes since Terraform waits for the EC2 instance to become available.

Inspect state

- When you applied your configuration, Terraform wrote data into a file called `terraform.tfstate`. Terraform stores the IDs and properties of the resources it manages in this file, so that it can update or destroy those resources going forward.

- The Terraform state file is the only way Terraform can track which resources it manages, and often contains sensitive information, so you must store your state file securely and restrict access to only trusted team members who need to manage your infrastructure.

- Inspect the current state using `terraform show`.

Managing state



Terraform has a built-in command called terraform state for advanced state management.



Use the list subcommand to list of the resources in your project's state.



Use --help subcommand to see other subcommands.

Terraform Lab: Change infrastructure

- We created a single EC2 instance on AWS with Terraform. Next, we will modify that resource, and learn how to apply changes to our Terraform project.

- Infrastructure is continuously evolving. IaC tools, such as Terraform, help you manage changes. As you change Terraform configurations, Terraform builds an execution plan that only modifies what is necessary to reach your desired state.

- When using Terraform in production, it is recommended to use a version control system to manage your configuration files and store your state in a remote backend such as Terraform Cloud or Terraform Enterprise.

Update configuration

- Now update the `ami` of your instance.

- Change the `aws_instance.app_server` resource under the provider block in `main.tf` by replacing the current AMI ID with a new one.
  - ami-08d70e59c07c61a3a

- This update changes the AMI to an Ubuntu 16.04 AMI. The AWS provider knows that it cannot change the AMI of an instance after it has been created, so Terraform will destroy the old instance and create a new one.

Apply changes

- After changing the configuration, run `terraform apply` again to see how Terraform will apply this change to the existing resources.
- The prefix `-/+` means that Terraform will destroy and recreate the resource, rather than updating it in-place. Terraform can update some attributes in-place (indicated with the `~` prefix), but changing the AMI for an EC2 instance requires recreating it. Terraform handles these details for you, and the execution plan displays what Terraform will do.
- Additionally, the execution plan shows that the AMI change is what forces Terraform to replace the instance. Using this information, you can adjust your changes to avoid destructive updates if necessary.
- Once again, Terraform prompts for approval of the execution plan before proceeding. Answer yes to execute the planned steps.

Destroy

- Once you no longer need infrastructure, you may want to destroy it to reduce your security exposure and costs.
- The `terraform destroy` command terminates resources managed by your Terraform project. This command is the inverse of `terraform apply` in that it terminates all the resources specified in your Terraform state. It does not destroy resources running elsewhere that are not managed by the current Terraform project.
- Just like with apply, Terraform determines the order to destroy your resources. In this case, Terraform identified a single instance with no other dependencies, so it destroyed the instance. In more complicated cases with multiple resources, Terraform will destroy them in a suitable order to respect dependencies.

Terraform Lab: Define input variables

- The examples so far have used hard-coded values. Terraform configurations can include *variables* to make your configuration more dynamic and flexible.
- **Task**: Set the instance name with a variable
  - Create a new file called `variables.tf` with a block defining a new `instance_name` variable. (See pages: Write configuration – variable block)
  - In `main.tf`, update the `aws_instance` resource block to use the new variable. The `instance_name` variable block will default to its default value ("ExampleAppServerInstance") unless you declare a different value. (See pages: Write configuration – using input variable values)

Write configuration – variable block

- Each input variable accepted by a module must be declared using a `variable` block
- The label after the `variable` keyword is a name for the variable, which must be unique among all variables in the same module.
- This name is used to assign a value to the variable from outside and to reference the variable's value from within the module.
- The name of a variable can be any valid identifier except the following: *source, version, providers, count, for_each, lifecycle, depends_on, locals.*

```
variable "instance_name" {
  description = "Value of the Name tag for
  the EC2 instance"
  type        = string
  default     = "ExampleAppServerInstance"
}
```

Write configuration – variable block

- Arguments – default values
- If a default value present, the variable is considered to be optional and the default value will be used if no value is set when calling the module or running Terraform.
- The default argument requires a literal value and cannot reference other objects in the configuration.

```
variable "instance_name" {
  description = "Value of the Name
  tag for the EC2 instance"
  type        = string
  default     =
  "ExampleAppServerInstance"
}
```

Write configuration – variable block

- Arguments – type constraints
- The `type` argument in a `variable` block allows you to restrict the type of value that will be accepted as the value for a variable. If no type constraint is set then a value of any type is accepted.
- While type constraints are optional, we recommend specifying them; they can serve as helpful reminders for users of the module, and they allow Terraform to return a helpful error message if the wrong type is used.

```
variable "instance_name" {
  description = "Value of the Name tag
  for the EC2 instance"
  type        = string
  default     =
  "ExampleAppServerInstance"
}
```

Write configuration – variable block

- Arguments – description
- Because the input variables of a module are part of its user interface, you can briefly describe the purpose of each variable using the optional `description` argument
- The description should concisely explain the purpose of the variable and what kind of value is expected.
- This description string might be included in documentation about the module, and so it should be written from the perspective of the user of the module rather than its maintainer.
- For commentary for module maintainers, use comments

```
variable "instance_name" {
  description = "Value of the Name tag for the EC2
   instance"
  type        = string
  default     = "ExampleAppServerInstance"
}
```

Write configuration – comments

begins a single-line comment, ending at the end of the line.

// also begins a single-line comment, as an alternative to #.

/* and */ are start and end delimiters for a comment that might span over multiple lines.

Write configuration – using input variable values

- Within the module that declared a variable, its value can be accessed from within expressions as `var.<NAME>`, where `<NAME>` matches the label given in the declaration block
- Input variables are *created* by a `variable` block, but you *reference* them as attributes on an object named `var`
- The value assigned to a variable can only be accessed in expressions within the module (see pages: Write configuration – modules) where it was declared

Update main.tf

Previous argument in tags:

Name = "ExampleAppServerInstance"

Updated argument in tags:

Name = var.instance_name

Write configuration – modules



Modules are containers for multiple resources that are used together. A module consists of a collection of .tf and/or .tf.json files kept together in a directory.



Modules are the main way to package and reuse resource configurations with Terraform.



Every Terraform configuration has at least one module, known as its *root module*, which consists of the resources defined in the .tf files in the main working directory.

Apply configuration changes

- Apply the configuration. Respond to the confirmation prompt with a yes

- Now apply the configuration again, this time overriding the default instance name by passing in a variable using the `-var` flag.

- Terraform will update the instance's `Name` tag with the new name. Respond to the confirmation prompt with `yes`.

```
terraform apply -var "instance_name=YetAnotherName"
```

Output values

- Output values make information about your infrastructure available on the command line and can expose information for other Terraform configurations to use. Output values are similar to return values in programming languages

- Resource instances managed by Terraform each export attributes whose values can be used elsewhere in configuration. Output values are a way to expose some of that information to the user of your module.

Terraform Lab: Outputs

- In the previous lab, we used an input variable to parameterize our Terraform configuration. In this lab, we will use output values to present useful information to the Terraform user.

- **Task**: Output EC2 instance configuration
  - Create a file called `outputs.tf` in your `terraform-lab-aws-instance` directory.
  - Add the configuration `outputs.tf` to define outputs for your EC2 instance's ID and IP address. (See pages: Write configuration: outputs)

Write configuration – outputs

- Each output value exported by a module must be declared using an `output` block
- The label immediately after the `output` keyword is the name, which must be a valid identifier.
- The `value` argument takes an expression whose result is to be returned to the user.
- In this example, the expression refers to the `id` and `public_ip` attributes exposed by an `aws_instance.app_server` resource

```
output "instance_id" {
  description = "ID of the EC2 instance"
  value       = aws_instance.app_server.id
}


output "instance_public_ip" {
  description = "Public IP address of the EC2 instance"
  value       = aws_instance.app_server.public_ip
}
```

Write configuration – outputs

- Because the output values of a module are part of its user interface, you can briefly describe the purpose of each value using the optional `description` argument
- The description should concisely explain the purpose of the output and what kind of value is expected.
- This description string might be included in documentation about the module, and so it should be written from the perspective of the user of the module rather than its maintainer.

```
output "instance_id" {
  description = "ID of the EC2
  instance"
  value       =
  aws_instance.app_server.id
}


output "instance_public_ip" {
  description = "Public IP address of
  the EC2 instance"
  value       =
  aws_instance.app_server.public_ip
}
```

Inspect output values

- You must apply this configuration before you can use these output values. Apply your configuration now. Respond to the confirmation prompt with `yes`.
- Terraform prints output values to the screen when you apply your configuration. Query the outputs with the `terraform output` command
- You can use Terraform outputs to connect your Terraform projects with other parts of your infrastructure, or with other Terraform projects

Cleanup

- Destroy your infrastructure with `terraform destroy`. Respond to the confirmation prompt with `yes`

